

```

import copy

def find_blank(state):
    """Finds the row and column of the blank space (0)."""
    for r in range(3):
        for c in range(3):
            if state[r][c] == 0:
                return r, c
    return None

def get_neighbors(state):
    """Generates valid next states by moving the blank space Up, Down, Left, or Right."""
    neighbors = []
    r, c = find_blank(state)

    # Define possible moves: (row_change, col_change, move_name)
    moves = [(-1, 0, 'Up'), (1, 0, 'Down'), (0, -1, 'Left'), (0, 1, 'Right')]

    for dr, dc, move in moves:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            new_state = copy.deepcopy(state)
            # Swap the blank space with the neighbor
            new_state[r][c], new_state[nr][nc] = new_state[nr][nc], new_state[r][c]
            neighbors.append((new_state, move))

    return neighbors

def state_to_tuple(state):
    """Converts a 2D list to a tuple of tuples for hashing."""
    return tuple(tuple(row) for row in state)

def print_state(state):
    """Prints the puzzle grid cleanly."""
    for row in state:
        print(" ".join(str(x) if x != 0 else "_" for x in row))
    print()

def dfs_with_limit(current_state, goal_state, path, visited, depth, limit):
    """Depth-Limited Search helper function."""
    if current_state == goal_state:
        return path
    if depth >= limit:
        return None

    state_tuple = state_to_tuple(current_state)
    visited.add(state_tuple)

    for neighbor_state, move in get_neighbors(current_state):
        if state_to_tuple(neighbor_state) not in visited:
            result = dfs_with_limit(neighbor_state, goal_state, path + [move], visited, depth + 1, limit)
            if result is not None:
                return result

    visited.remove(state_tuple) # Backtrack
    return None

def iddfs_8_puzzle(start_state, goal_state, max_limit=30):
    """Solves the 8-puzzle using Iterative Deepening DFS."""
    for limit in range(max_limit + 1):
        visited = set()
        result = dfs_with_limit(start_state, goal_state, [], visited, 0, limit)
        if result is not None:
            return result
    return None

def get_user_input(prompt_name):
    """Helper to take a 3x3 matrix input from the user."""
    print(f"\n--- Enter the {prompt_name} State (Use 0 for blank) ---")
    print("Enter 3 numbers per row, separated by spaces (e.g., 1 2 3):")
    state = []
    for i in range(3):
        while True:
            try:
                row = list(map(int, input(f"Row {i+1}: ").split()))

```

```

        if len(row) != 3:
            print("Error: You must enter exactly 3 numbers.")
            continue
        state.append(row)
        break
    except ValueError:
        print("Error: Please enter numbers only.")
    return state

# --- Execution ---
if __name__ == "__main__":
    # Print student details first
    print("=====")
    print("Name: HRITHIKA RAJESH")
    print("USN: 1BM24CS120")
    print("=====\n")

    print("=== 8-PUZZLE DFS SOLVER ===")

    # Get custom states from user
    initial_state = get_user_input("INITIAL")
    goal_state = get_user_input("GOAL")

    print("\nInitial State:")
    print_state(initial_state)

    print("Target Goal State:")
    print_state(goal_state)

    print("Searching for a path...")
    # Limit maximum depth to 20 to avoid heavy computations
    solution_path = iddfs_8_puzzle(initial_state, goal_state, max_limit=20)

    if solution_path is not None:
        print(f"Solution Found in {len(solution_path)} steps!")
        print("Moves:", " -> ".join(solution_path) if solution_path else "Already at goal state!")
    else:
        print("❌ No solution found within a reasonable depth limit.")
        print("Tip: DFS is highly inefficient for random configs. Try states that are only 2-5 moves apart.")

```

```

=====
Name: HRITHIKA RAJESH
USN: 1BM24CS120
=====

=== 8-PUZZLE DFS SOLVER ===

--- Enter the INITIAL State (Use 0 for blank) ---
Enter 3 numbers per row, separated by spaces (e.g., 1 2 3):
Row 1: 2 8 3
Row 2: 1 6 4
Row 3: 7 0 5

--- Enter the GOAL State (Use 0 for blank) ---
Enter 3 numbers per row, separated by spaces (e.g., 1 2 3):
Row 1: 1 2 3
Row 2: 8 0 4
Row 3: 7 6 5

Initial State:
2 8 3
1 6 4
7 _ 5

Target Goal State:
1 2 3
8 _ 4
7 6 5

Searching for a path...
Solution Found in 5 steps!
Moves: Up -> Up -> Left -> Down -> Right

```

